

# How the data-quality assessment of the OPERA archive was computed

Notes for the `minkowski-loss` project

23 September 2026

Part II added 3 October 2026

## Abstract

These notes describe the data-quality audit run on 23 September 2026 over the raw OPERA radar archive: 745 days, 4.8 million tiles of  $128 \times 128$  pixels. They cover every step from the stored rain rate to the summary tables:

- what is read;
- how tiles are cut and keyed;
- how a per-pixel clutter climatology is built;
- the exact definition of each of the  $\sim 45$  per-tile features;
- the candidate artefact rules;
- the statistics in the summary;
- the split-leakage audit;
- the image gallery used for labelling.

Each quantity is defined mathematically and illustrated. A final part reports the first full-run results and how the code was checked.

The audit *describes* the data. It deletes nothing, and its rule thresholds are starting guesses to be calibrated against labels.

Part II (added 3 October) records what followed:

- two corrections to Part I (a mirrored DEM, a capped source) and the product break from ODYSSEY to NIMBUS;
- the v2 cleaning, which repairs pixels and rejects few tiles;
- two audit flags, temporal support and climatological range rings;
- a per-radar quality ranking over the whole archive;
- a validation of every rule against 1,735 independent rain gauges.

## Contents

|          |                                         |          |
|----------|-----------------------------------------|----------|
| <b>I</b> | <b>The audit of 23 September 2026</b>   | <b>2</b> |
| <b>1</b> | <b>What the assessment is for</b>       | <b>2</b> |
| <b>2</b> | <b>The data it reads</b>                | <b>3</b> |
| <b>3</b> | <b>Pipeline overview</b>                | <b>4</b> |
| <b>4</b> | <b>Tiles and keys</b>                   | <b>5</b> |
| <b>5</b> | <b>Stage 1: the clutter climatology</b> | <b>5</b> |

|           |                                                                                          |           |
|-----------|------------------------------------------------------------------------------------------|-----------|
| <b>6</b>  | <b>Stage 2: the tile features</b>                                                        | <b>6</b>  |
| 6.1       | Intensity and concentration . . . . .                                                    | 6         |
| 6.2       | Speckle, spikes and gradients . . . . .                                                  | 6         |
| 6.3       | Connected components and the spoke detector . . . . .                                    | 7         |
| 6.4       | Relation to the coarse input . . . . .                                                   | 7         |
| 6.5       | What the current declutter step does . . . . .                                           | 7         |
| 6.6       | Terrain and sea . . . . .                                                                | 7         |
| 6.7       | OPERA's quality index . . . . .                                                          | 8         |
| 6.8       | Persistence in time . . . . .                                                            | 8         |
| 6.9       | Static clutter at the peak . . . . .                                                     | 8         |
| <b>7</b>  | <b>Stage 4a: candidate rules</b>                                                         | <b>8</b>  |
| <b>8</b>  | <b>Stage 4b: the summary statistics</b>                                                  | <b>9</b>  |
| <b>9</b>  | <b>Stage 3: the split audit</b>                                                          | <b>10</b> |
| <b>10</b> | <b>Stage 5: the gallery and the labelling loop</b>                                       | <b>10</b> |
| <b>11</b> | <b>First full run: what it shows</b>                                                     | <b>11</b> |
| <b>12</b> | <b>How the code was checked</b>                                                          | <b>11</b> |
| <b>13</b> | <b>Limitations</b>                                                                       | <b>13</b> |
| <b>II</b> | <b>After the audit: the v2 screen and its validation (28 September – 3 October 2026)</b> | <b>13</b> |
| <b>14</b> | <b>Corrections to Part I</b>                                                             | <b>14</b> |
| <b>15</b> | <b>The v2 cleaning</b>                                                                   | <b>14</b> |
| <b>16</b> | <b>Audit flags: temporal support and range rings</b>                                     | <b>15</b> |
| <b>17</b> | <b>Consistently bad radars</b>                                                           | <b>16</b> |
| <b>18</b> | <b>Validation against rain gauges</b>                                                    | <b>17</b> |

## Part I

# The audit of 23 September 2026

## 1 What the assessment is for

The project's claims are about the upper tail of rain rate. Artefacts of the radar network concentrate in that tail: ground and sea clutter, interference spokes, range rings, one-frame bursts and decoding errors. A patch ranked by its maximum is therefore more likely to be an artefact the higher it ranks. The audit answers four questions before any screening decision is made:

1. Which artefact *signatures* are present, and how common are they as a function of intensity, year, season, time of day and location?

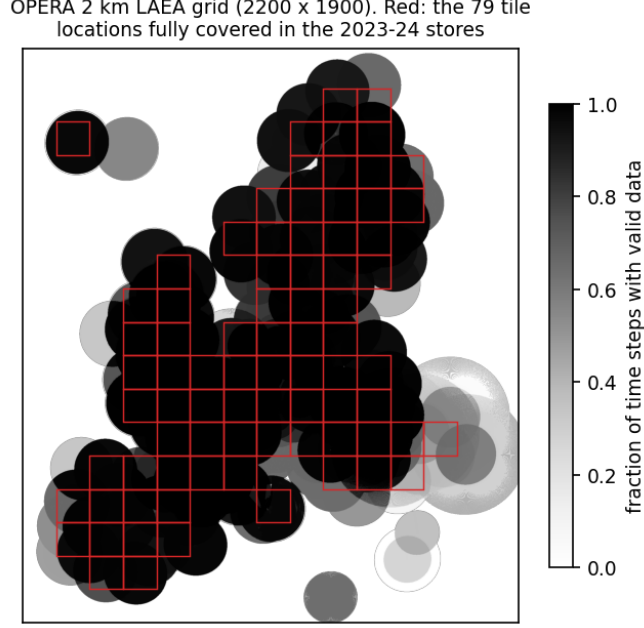


Figure 1: Fraction of time steps with valid data at each pixel over all 745 days (grey), and the tile locations that are fully covered in the 2023–24 stores (red). Tiles are  $128 \times 128$  pixels ( $256 \times 256$  km) on a stride-128 grid.

2. Which ones overlap, so that a single rule would suffice, and which ones are independent?
3. What does the current preprocessing (`filter_precip_bounds`) do to the tail?
4. How independent are the current train/validation/test splits?

Everything is computed on the *raw* stored field, before any filtering, and on the full downloaded archive, not only on the patches that entered training.

## 2 The data it reads

**Grid and time.** Each day is one zarr store `RAW_OPERA_DATA_DIR/YYYYMMDD/`. It holds a rain-rate array  $R_t(y, x)$  in mm/h, for  $t$  over the day’s 15-minute composites and  $(y, x)$  on the OPERA 2 km Lambert azimuthal equal-area grid of  $2200 \times 1900$  pixels. Pixels outside radar coverage are NaN, and “no echo” is 0. Stores written by the archive fetcher (`scripts/data/fetch_opera_archive.py`) also carry the OPERA total quality index  $Q_t(y, x) \in [0, 1]$ . Figure 1 shows the coverage.

**Two kinds of store.** The raw directory mixes two sources, and they differ in a way that matters.

- **Original 2023–24 stores** (Aug 2023 – Oct 2024, 442 days): the source of the current patch set. They hold the rate only, no QIND.
- **Fetchd archive stores** (2012–13 and test days in 2016, 2020 and July 2023, 303 days): written by the fetcher. They hold rate and QIND, with no alteration of values.

Figure 2 compares their tile maxima. The original stores *never exceed 150 mm/h*. Over their 2,853,816 fully covered tiles the maximum is exactly 150.0 and the 99.99th percentile is 149.94. The archive stores continue smoothly past 150 and reach  $\sim 10^5$  mm/h (decoding errors and clutter).

- So the 150 mm/h cut is present in the original raw source itself, not only in our preprocessing. Whether the source clipped or removed the values above 150 cannot be told from the stores.

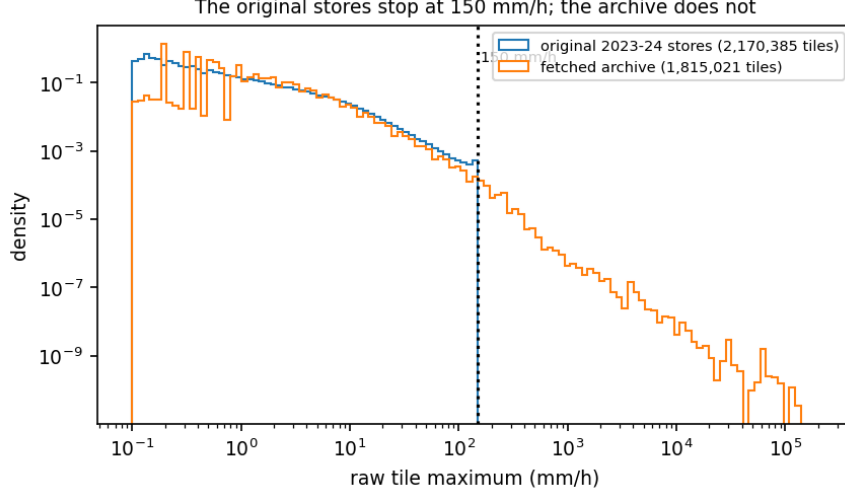


Figure 2: Distribution of raw tile maxima in the two kinds of store (tiles with  $\max > 0.1$  mm/h).

- Recovering real 2023–24 peaks would need those days re-fetched from the archive.

### 3 Pipeline overview

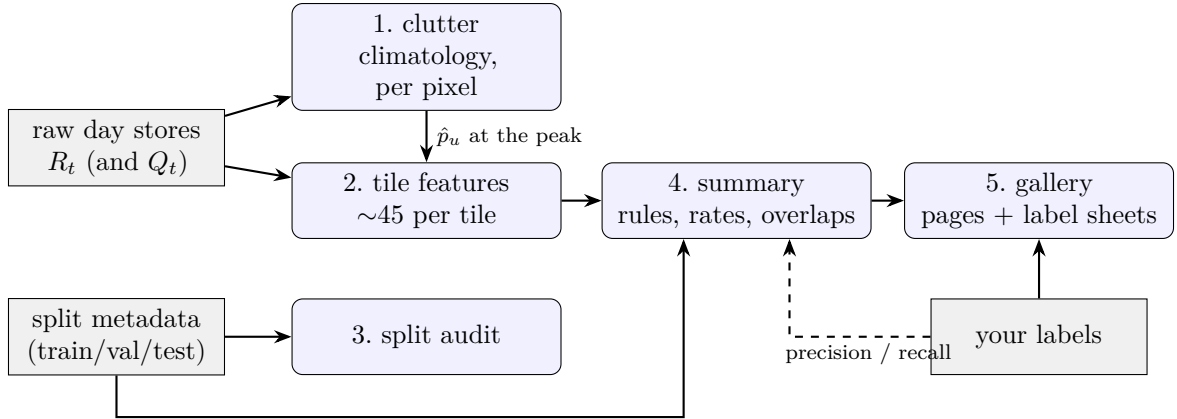


Figure 3: The five stages of `scripts/hpc/launch_data_quality.sh`. Dashed: the labelling loop, which closes after you label tiles.

Stage by stage (all scripts are in `scripts/data_quality/`; the feature definitions are in `src/data/quality.py`):

1. `clutter_climatology.py` counts, per pixel, how often each threshold is exceeded (§5).
2. `compute_tile_features.py` cuts every tile at every time step and computes the features (§6). Clutter frequencies from stage 1 are joined at the tile’s peak.
3. `audit_splits.py` measures the overlap of the current splits (§9).
4. `summarize_features.py` applies the candidate rules and aggregates (§7–8).
5. `patch_gallery.py` renders tiles for inspection and writes label sheets (§10).

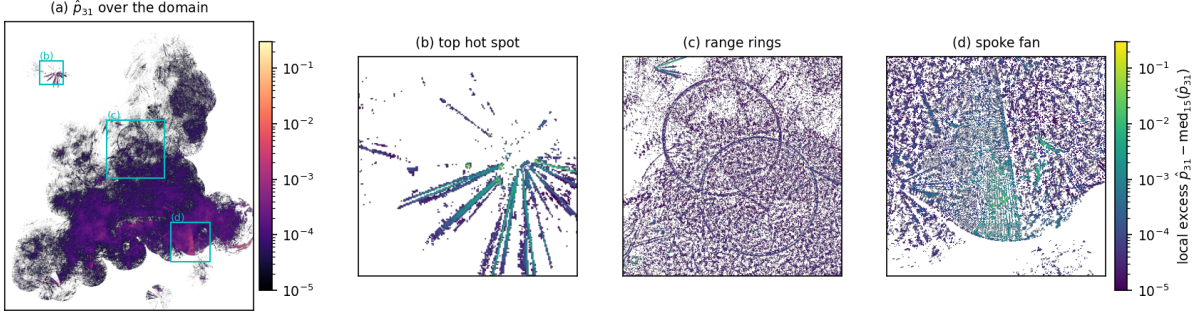


Figure 4: (a) Exceedance frequency of 31 mm/h over all 745 days. (b–d) Local excess  $e_{31}$  in three zooms. The top hot-spot pixel exceeds 31 mm/h in 23% of all time steps.

## 4 Tiles and keys

A tile is identified by its time stamp  $t$  and its lower-left pixel  $(r, c)$ , with  $r, c$  multiples of 128. It is the array

$$T(i, j) = R_t(r + i, c + j), \quad 0 \leq i, j < 128. \quad (1)$$

The tile grid, the key format `timestamp,row,col` and the orientation are exactly those of `scripts/preprocess/generate_metadata.py`. So every feature row joins one-to-one onto the existing patch metadata. A tile is kept when its coverage  $\text{cov}(T) = \frac{1}{128^2} \sum_{ij} \mathbf{1}[T(i, j) \text{ finite}]$  equals 1 (flag `--min_coverage`), which is the current patch definition.

*Remark* (Alignment check). Applying the pipeline’s own filter to a tile and taking the maximum reproduces the `max_precip` column of the metadata exactly (difference 0.0 on every joined test tile). The number of fully covered tiles in the original stores, 2,853,816, equals the total number of train + validation + test patches. Both checks confirm that the audit sees exactly the tiles the model was trained on.

## 5 Stage 1: the clutter climatology

A real storm crosses a given pixel a handful of times a year. Ground clutter, wind farms, sea clutter and interference light up the *same* pixels again and again. For every pixel and threshold  $u \in \{1, 31, 89, 150, 500\}$  mm/h, over all time steps of all days:

$$n(y, x) = \sum_t \mathbf{1}[R_t(y, x) \text{ finite}], \quad (2)$$

$$n_u(y, x) = \sum_t \mathbf{1}[R_t(y, x) \geq u], \quad \hat{p}_u(y, x) = \frac{n_u(y, x)}{\max(n(y, x), 1)}. \quad (3)$$

The estimate  $\hat{p}_u$  is the empirical exceedance frequency. Weather varies smoothly in space on these scales, while clutter is pixel-sharp. The *local excess*

$$e_u(y, x) = \hat{p}_u(y, x) - \text{med}_{15 \times 15} \hat{p}_u(y, x) \quad (4)$$

therefore removes the climatological background (a  $15 \times 15$  median filter, 30 km) and keeps what stands out from its surroundings. The script also stores yearly counts, the per-pixel maximum and the mean QIND, and it ranks the top-50 pixels by  $e_{31}$  (`clutter_hotspots.csv`). Figure 4 shows the result: sharp spokes radiating from individual radars, complete range rings, and a fan of interference.

The tile features join two numbers from this map (§6.9):  $\hat{p}_u$  at the tile’s peak pixel, and the maximum of  $\hat{p}_u$  over the tile.

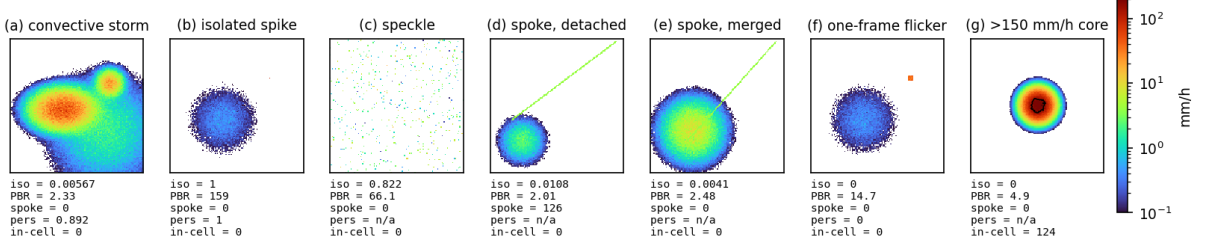


Figure 5: Synthetic tiles and the values the audit computes for them. **iso**: isolated fraction at 1 mm/h. **PBR**: peak/coarse-block ratio. **spoke**: spoke length in px. **pers**: persistence into  $t - 15$ . **in-cell**: pixels  $> 150$  mm/h inside a cell. Panel (e) shows a known limitation: a spoke that touches rain merges with it and is not detected.

## 6 Stage 2: the tile features

All features are computed on the raw tile  $T$ , with NaN treated as missing. The notation used throughout:

- $Z = T$  with  $\text{NaN} \rightarrow 0$ ;
- $M = \max Z$ , the tile maximum, attained at  $p^* = (i^*, j^*)$ ;
- $\mathcal{N}_8(p)$ , the eight neighbours of pixel  $p$  (edge pixels have fewer);
- $\delta = 0.1$  mm/h, the drizzle level;
- $D = 150$  mm/h, the current declutter threshold.

Figure 5 applies the actual feature code to synthetic tiles that isolate one signature each.

### 6.1 Intensity and concentration

$$M = \max_{ij} Z_{ij}, \quad \bar{T} = \text{mean}_{\text{finite}} T, \quad q_{99} = 99\text{th percentile}, \quad (5)$$

$$\text{wet} = \frac{1}{|T|} \#\{T \geq \delta\}, \quad N_u = \#\{T \geq u\} \quad (u \in \{1, 31, 53, 89, 150, 500\}), \quad (6)$$

$$\text{conc} = \bar{T}/M. \quad (7)$$

The *concentration* is near 0 when the tile's rain is carried by a few pixels. A real convective cell raises  $M$  and the wet area together, so *conc* falls only slowly with  $M$ . A single bright pixel drives it to  $\sim 10^{-4}$ .

### 6.2 Speckle, spikes and gradients

Let  $\text{nbmax}(p) = \max_{q \in \mathcal{N}_8(p)} Z_q$ , with  $\text{nbmean}$ ,  $\text{nbmed}$  and  $\text{nbmin}$  defined likewise.

- **Isolated fraction** at level  $u \in \{0.1, 1, 10\}$ : the share of pixels  $\geq u$  with no neighbour  $\geq u$ ,

$$\text{iso}_u = \frac{\#\{p : Z_p \geq u, \text{nbmax}(p) < u\}}{\#\{p : Z_p \geq u\}}. \quad (8)$$

Rain fields are spatially coherent at 2 km, so  $\text{iso}_u$  is small for weather and large for speckle (Fig. 5c: 0.82).

- **Spikes**: pixels  $\geq 10$  mm/h whose brightest neighbour is below a quarter of them,  $\mathcal{S} = \{p : Z_p \geq 10, \text{nbmax}(p) < \frac{1}{4}Z_p\}$ . The features are  $n_{\text{spikes}} = |\mathcal{S}|$  and the *spike mass fraction*  $\sum_{\mathcal{S}} Z / \sum Z$ .
- **Peak-to-neighbour ratio**:  $M / (\text{nbmean}(p^*) + \delta)$ .
- **Maximum gradient**: the largest absolute difference between 4-neighbours,  $\max |T_{i+1,j} - T_{ij}| \vee |T_{i,j+1} - T_{ij}|$ , in mm/h per 2 km.

### 6.3 Connected components and the spoke detector

Components are the 8-connected sets of  $\{Z \geq u\}$  for  $u \in \{1, 31\}$ . The features are the number of components and the size of the largest.

Interference spokes are long, thin and straight. For each component  $C$  with  $|C| \geq 30$  px, take the covariance of its pixel coordinates,

$$\Sigma_C = \frac{1}{|C|} \sum_{p \in C} (p - \bar{p})(p - \bar{p})^\top, \quad \lambda_{\max} \geq \lambda_{\min} \text{ its eigenvalues.} \quad (9)$$

The *elongation*  $\epsilon_C = 1 - \sqrt{\lambda_{\min}/\lambda_{\max}}$  is 0 for a disc and tends to 1 for a line. A uniform segment of length  $L$  has variance  $L^2/12$  along its axis, so

$$L_C = \sqrt{12 \lambda_{\max}} \quad (10)$$

estimates the component's length. The features are  $\max_C \epsilon_C$  and the *spoke length*  $\max\{L_C : \epsilon_C > 0.9\}$  (Fig. 5d: 126 px).

*Remark.* A spoke crossing a rain area is part of a larger, non-elongated component and is invisible to this detector (Fig. 5e). Detecting it would need radar-site geometry (azimuth from the site), which the audit does not have.

### 6.4 Relation to the coarse input

The model is conditioned on a  $10 \times 10$  coarse field made by `torch.nn.functional.adaptive_avg_pool2d` of the *filtered* tile. With  $n = 128$  and  $m = 10$ , bin  $k$  covers pixels

$$[\lfloor kn/m \rfloor, \lceil (k+1)n/m \rceil), \quad (11)$$

which are 13 or 14 pixels wide and overlap by one (Fig. 6). The audit reproduces these bins exactly (unit-tested against torch). Let  $\bar{B}(p^*)$  be the mean of the coarse block containing the peak. Then

$$\text{PBR} = \frac{M}{\bar{B}(p^*) + \delta}. \quad (12)$$

A lone pixel in an otherwise dry block gives  $\text{PBR} \approx$  the block area ( $\sim 170$ ). A convective core, whose surroundings are also wet, gives single digits. A large PBR means the target contains a peak that the input gives no hint of, which is exactly what makes artefacts harmful to a super-resolution model.

### 6.5 What the current declutter step does

`filter_precip_bounds` sets every value outside  $[\delta, D]$  to zero, and NaN to zero. The audit counts

$$n_{>D} = \#\{p : Z_p > D\}, \quad n_{>D}^{\text{cell}} = \#\{p : Z_p > D, \text{nbmed}(p) \geq 10\}, \quad (13)$$

the second being the pixels above 150 that sit *inside* a wet area. Zeroing those punches a hole in the storm (Fig. 7, top). It also counts pre-existing holes,  $\#\{p : Z_p < \delta, \text{nbmin}(p) \geq 10\}$ .

### 6.6 Terrain and sea

With the DEM on the same grid, sea is  $\{\text{DEM} \leq 0\} \cup \{\text{DEM missing}\}$ . The features are:

- the sea fraction and mean elevation;
- whether the peak lies over sea;
- the share of wet ( $\geq 1$  mm/h) pixels that lie over sea.

Sea clutter shows up as intense peaks over sea in coarse blocks that are almost dry.

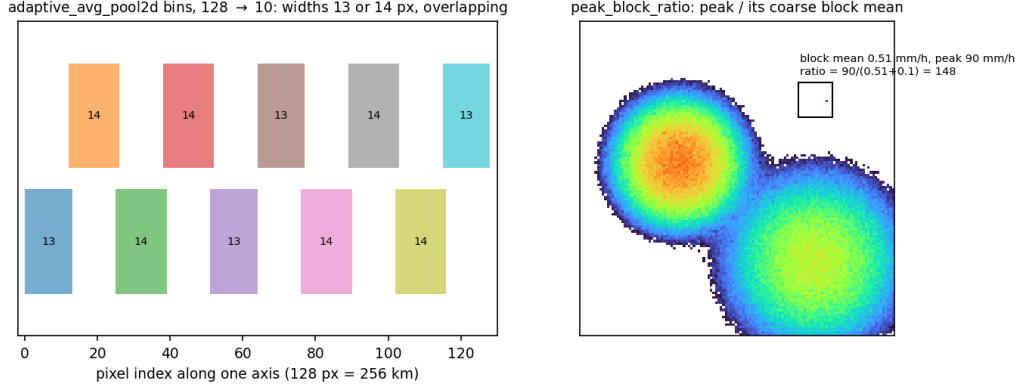


Figure 6: Left: the ten adaptive-pooling bins along one axis. Right: the peak/coarse-block ratio for an isolated 90 mm/h pixel.

## 6.7 OPERA’s quality index

Where the store has QIND:

- its coverage  $\#\{Q \text{ finite}\}/\#\{T \text{ finite}\}$ ;
- the mean of  $Q$  over all pixels, over wet pixels and over pixels  $\geq 31$  mm/h;
- the share of  $\geq 31$  mm/h pixels with  $Q < 0.5$ ;
- $Q(p^*)$ .

The QIND coverage is itself informative: in the archive it is missing on most of the covered area.

## 6.8 Persistence in time

Real storms persist and move. At 15-minute resolution a cell moving at  $\leq 50$  km/h travels  $\leq 12.5$  km, about 6 pixels. With  $W$  the  $13 \times 13$  window centred on  $p^*$  ( $\pm 6$  px), and  $R_{t\pm 1}$  the neighbouring frames of the same tile:

$$\text{pers}_{\pm} = \frac{\max_W R_{t\pm 1}}{M}. \quad (14)$$

A persistent storm gives values near 1: the median for tail tiles with a large core in 2024 is 0.88. A one-frame burst gives  $\approx 0$  in both directions (Fig. 5f).

Values well above 1 are also informative: the neighbouring frame holds something much brighter nearby, typically a burst in *that* frame. The Pearson correlation of  $\log(1 + R_t)$  with  $\log(1 + R_{t-1})$  over the tile is recorded as `corr_prev`. The first and last frames of a day have no neighbour across the day boundary, and their persistence is missing.

## 6.9 Static clutter at the peak

From stage 1:  $\hat{p}_u(p^*)$  at the tile’s peak pixel, and  $\max_T \hat{p}_u$  over the tile, for each  $u$ . A tile peaking on a pixel that exceeds 31 mm/h in more than 1% of all time steps is almost certainly peaking on clutter.

A caveat: this is only as good as the climatology. On a few days it flags recurring *weather* too. Over 745 days it isolates the static artefacts.

## 7 Stage 4a: candidate rules

Each rule turns features into a flag for one signature. **The thresholds are first guesses**, to be calibrated against labels (§10).

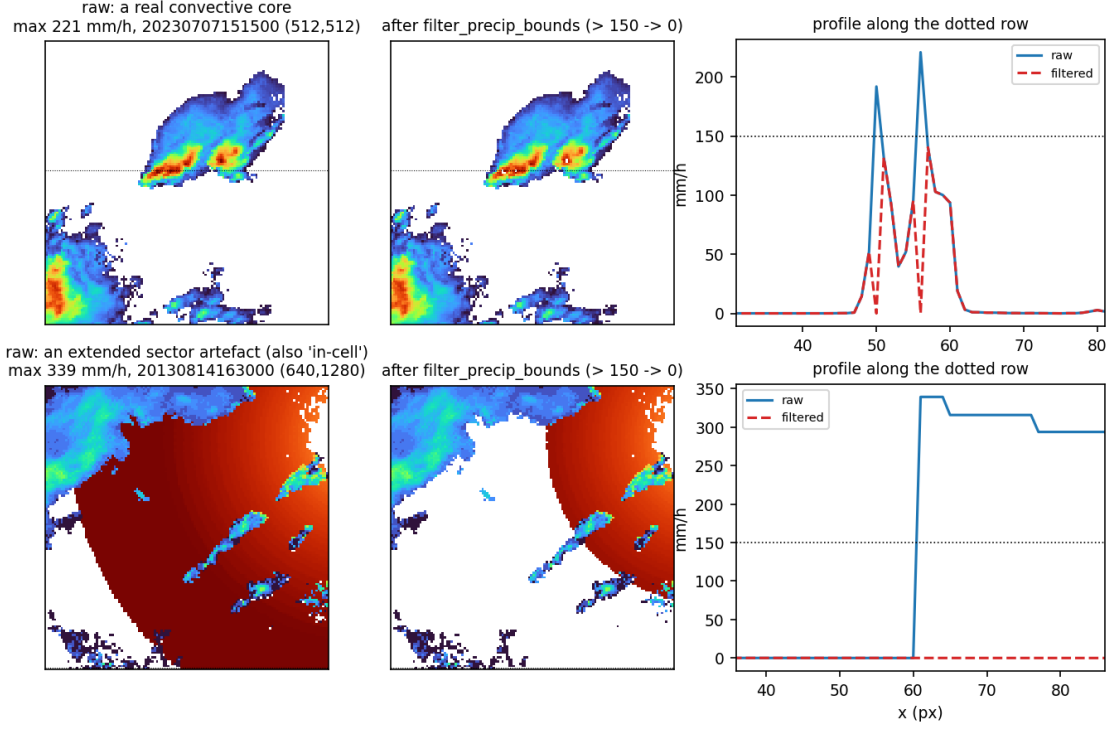


Figure 7: Top: a real convective core from the fetched July 2023 data. Its two peaks above 150 mm/h are set to 0 by the current filter, leaving holes at the storm’s centre. Bottom: an extended high-value sector artefact. It also passes the “inside a cell” test, because its neighbours are high too. The in-cell count therefore identifies holes in real storms only together with the other features.

| rule               | flag condition                                                                             |
|--------------------|--------------------------------------------------------------------------------------------|
| spike              | $n_{\text{spikes}} > 0$ and spike mass fraction $> 0.2$                                    |
| speckle            | $\text{iso}_1 > 0.5$ and $N_1 < 200$                                                       |
| isolated_peak      | $M \geq 10$ and $\text{PBR} > 60$                                                          |
| flicker            | $M \geq 10$ and $\max(\text{pers}_-, \text{pers}_+) < 0.2$                                 |
| sea_clutter        | $M \geq 10$ , peak over sea, and $\bar{B}(p^*) < 0.5$                                      |
| spoke              | spoke length $> 40$ px (80 km)                                                             |
| static_clutter     | $M \geq 31$ and $\hat{p}_{31}(p^*) > 0.01$                                                 |
| qind_low           | $M \geq 10$ and $Q(p^*) < 0.3$                                                             |
| declutter_isolated | $n_{>D} > n_{>D}^{\text{cell}}$ (pixels $> 150$ outside any wet area)                      |
| unphysical         | $M > 500$ mm/h                                                                             |
| declutter_hole     | $n_{>D}^{\text{cell}} > 0$ : not an artefact flag; the tiles the current filter would hole |

**Missing inputs are not “no”.** A rule whose inputs are missing gives NaN, not 0. Examples are `qind_low` on stores without QIND, and `flicker` at the first frame of a day. Rates are then computed over tiles where the rule is defined. `any_flag` is 1 if any defined rule fires, and missing only if no rule is defined.

## 8 Stage 4b: the summary statistics

**Strata.** Tiles are grouped by raw maximum  $M$  into  $[0, 0.1)$ ,  $[0.1, 1)$ ,  $[1, 10)$ ,  $[10, 31)$ ,  $[31, 53)$ ,  $[53, 89)$ ,  $[89, 150)$ ,  $[150, 500)$  and  $[500, \infty)$  mm/h. The *tail* is  $M \geq 31$  mm/h, the POT level of the project’s tail protocol. Stratifying by the raw maximum defines the tail before anything is

removed.

**Flag rates.** For a group  $G$  and rule  $k$ ,  $\hat{\pi}_k(G) = \frac{1}{|G_k|} \sum_{G_k} f_k$  over the tiles  $G_k \subseteq G$  where  $f_k$  is defined. The binomial standard error  $\sqrt{\hat{\pi}(1 - \hat{\pi})/|G_k|}$  understates the true uncertainty: tiles of the same storm and the same hour are strongly correlated. Treat differences below a few percentage points as noise until they are recomputed with day-level resampling.

**The key diagnostic.** In a clean archive, artefact signatures would become *rarer* towards the tail, because real extremes are large coherent storms. A flag rate that rises with  $M$  is therefore evidence of contamination, whatever the rules' individual precision.

**Overlap.** Rules  $k, l$  are compared by the Jaccard index over the tail,  $J_{kl} = |F_k \cap F_l| / |F_k \cup F_l|$ , where  $F_k$  is the set of flagged tiles. Small off-diagonal values mean the rules catch different things, so a screen needs their union.

**Breakdowns and outputs.** Rates are also broken down by year, month, hour (UTC), current split and tile location. The summary lists every flagged tail tile (`flagged_tail.csv`, 216,714 tiles) for the gallery.

**Against labels.** Given a sheet of labelled tiles (artefact / real / unsure), each rule gets

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN} \quad (15)$$

over tiles labelled artefact or real. Labels must come from a *random* draw of the tail as well as from flagged tiles. Otherwise precision cannot be separated from the base rate.

## 9 Stage 3: the split audit

`split_metadata.py` shuffles individual patches, so the same storm can be in train and test. For each test (and validation) patch  $(t, r, c)$  the audit asks whether the train split contains:

- *same time*: any patch at the same  $t$ ;
- *same tile within 1 h*: a patch at  $(t', r, c)$  with  $|t' - t| \leq 60$  min (found by binary search in the tile's sorted train times);
- *adjacent tile, same time*: a patch at  $(t, r \pm 128, c \pm 128)$ .

| query | subset              | $n$     | same time | same tile $\pm 1$ h | adjacent, same $t$ |
|-------|---------------------|---------|-----------|---------------------|--------------------|
| test  | all                 | 285,383 | 100.0%    | 100.0%              | 98.2%              |
| test  | patch max $\geq 31$ | 25,591  | 100.0%    | 100.0%              | 99.4%              |
| test  | patch max $\geq 89$ | 6,349   | 100.0%    | 100.0%              | 99.3%              |

Validation gives the same numbers. Every current test score is therefore an interpolation score within storms seen in training.

## 10 Stage 5: the gallery and the labelling loop

`patch_gallery.py` renders one row per tile (Fig. 8):

- the raw frames at  $t - 15$ ,  $t$  and  $t + 15$  min;
- the target after the current filter, where holes show as white;
- the coarse input as the model sees it;
- the DEM and QIND.

The row header carries the key, the raw maximum and the rules that fire. The selections are:

- **top\_max**: the highest maxima;
- **rule:<name>**: the tiles each rule flags;
- **unflagged**: tail tiles no rule flags, to find misses;
- **random**: an unbiased draw from the tail, needed for precision and recall.

Each page set comes with `labels.csv`. Fill its `label` column and pass the file to `summarize_features.py --labels` to close the loop in Fig. 3.

## 11 First full run: what it shows

The full run took about 4 hours with 6 workers on 8 cores. It covered 745 days (2012-09-04 to 2024-10-30) and 4,804,051 tiles, of which 413,654 (8.61%) are in the tail. The DEM-derived features and the `sea_clutter` rates were computed on a mirrored DEM (§14).

1. **The flag rate rises with intensity** (Fig. 9a). Any-flag is 18% of tiles at 1–10 mm/h, 47% at 31–53, 54% at 89–150, and  $\approx 100\%$  above 150. That is the contamination signature of §8.
2. **Old years are dirtier** (Fig. 9b). 92% of 2012 tail tiles are flagged, 75% in 2013, and 42% and 37% in 2023 and 2024. The QIND-based rule exists only for the fetched years.
3. **A diurnal cycle** (Fig. 9c). The tail flag rate is  $\approx 61\%$  at night and  $\approx 43\%$  mid-afternoon. Two effects combine: nighttime clutter (anomalous propagation under stable layers) and afternoon convection adding real tail tiles.
4. **The rules are complementary**. Most tail Jaccard indices are below 0.3; the largest is `isolated_peak` with `flicker` (0.36). No single rule is a screen.
5. **Static clutter is real and located** (Fig. 4). A few tile locations have 85–97% of their tail tiles flagged. The worst is (1024, 640).
6. **The declutter step touches 5.7% of tail tiles**. It would hole real cells in 7,867 of them, all in the fetched years, since the original stores stop at 150.

## 12 How the code was checked

- **Unit tests** (`tests/test_quality.py`, 8 tests). Each artefact feature must separate its synthetic artefact from a synthetic storm: spike against storm, the spoke’s length within 5%, flicker against advection, and declutter-in-cell against isolated. The coarse pooling is checked against `adaptive_avg_pool2d` to  $10^{-6}$ , and NaN and all-NaN tiles are handled.
- **Alignment**. The filtered tile maximum reproduces the metadata `max_precip` exactly. The original-store tile count equals the patch count.
- **Smoke tests** on 5 days before the full run, with visual inspection of the gallery pages.
- **Sanity checks on the full run**.
  - The distributions behave as expected by stratum (Fig. 10).
  - The persistence of large real cores has a median of 0.88.
  - Extreme persistence ratios ( $> 10^3$ , 65 tiles) trace to bursts in the neighbouring frame, not to arithmetic errors.

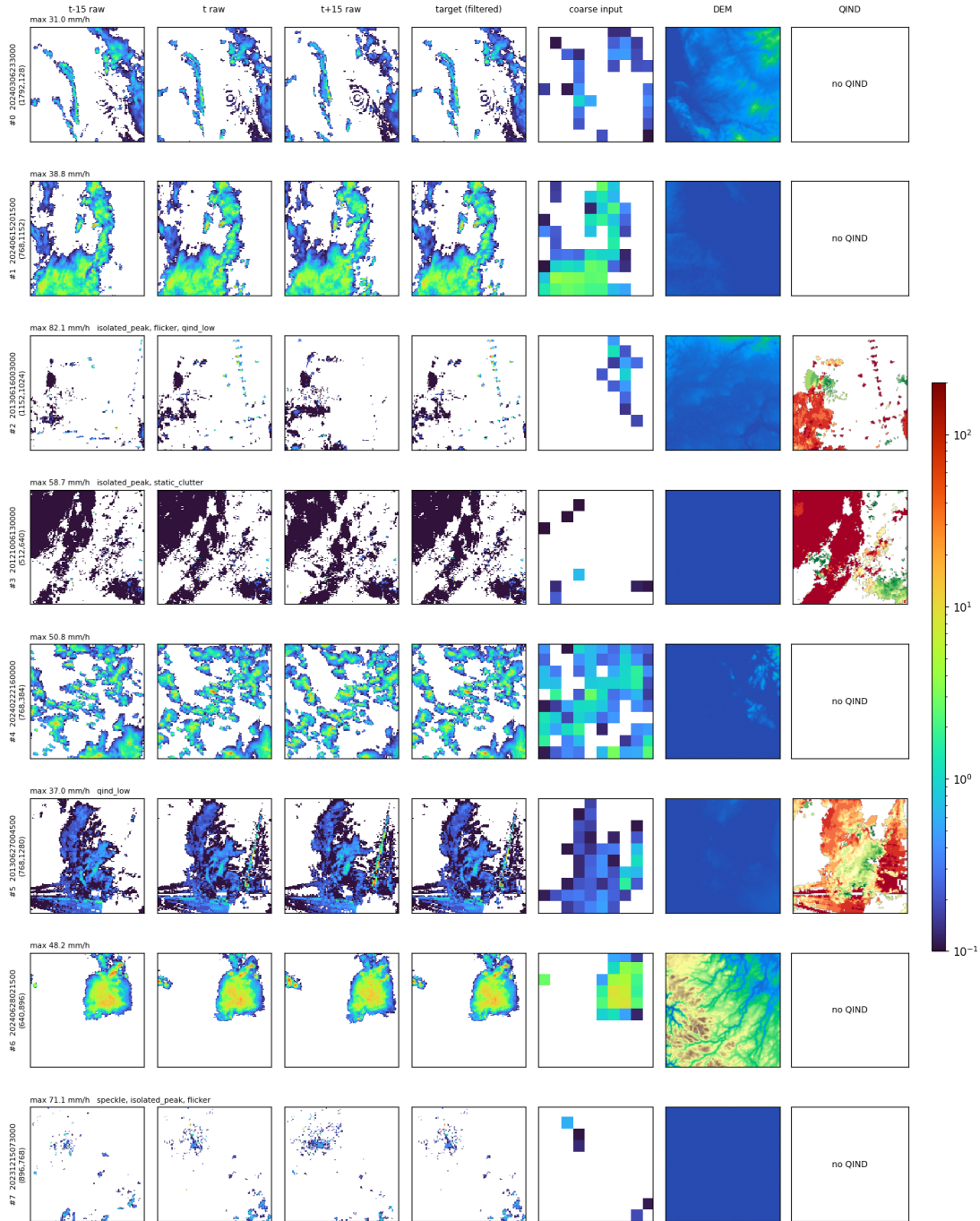


Figure 8: First page of the random tail draw. Rows 1, 4 and 6 are real storms with no rule firing. Rows 2, 3 and 7 are flagged artefacts. Row 0 is a real storm with a ring artefact that no rule catches. Row 5 is a storm crossed by a spoke that merges with the rain. Only `qind_low` fires there; the spoke rule misses it (Fig. 5e).

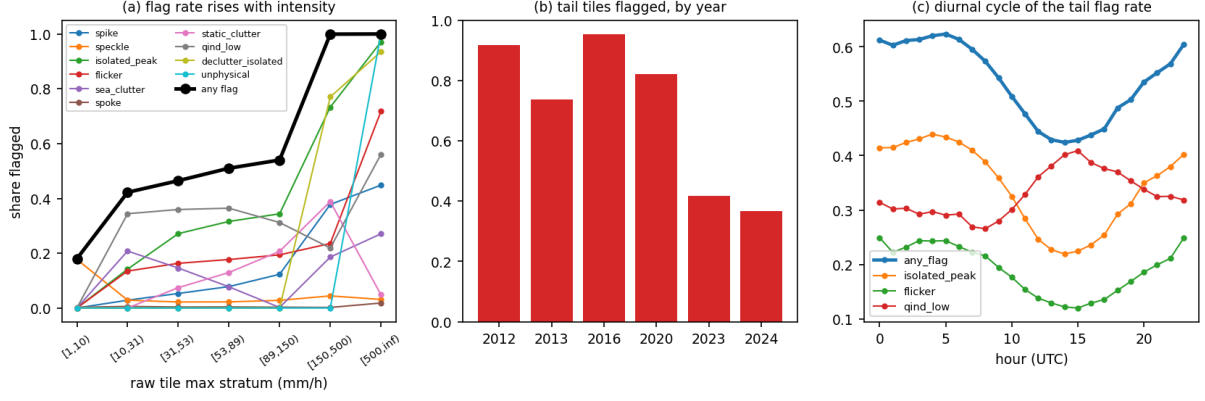


Figure 9: (a) Flag rate by raw-max stratum, per rule and for any rule. (b) Share of tail tiles flagged, by year. (c) Diurnal cycle of the tail flag rate.

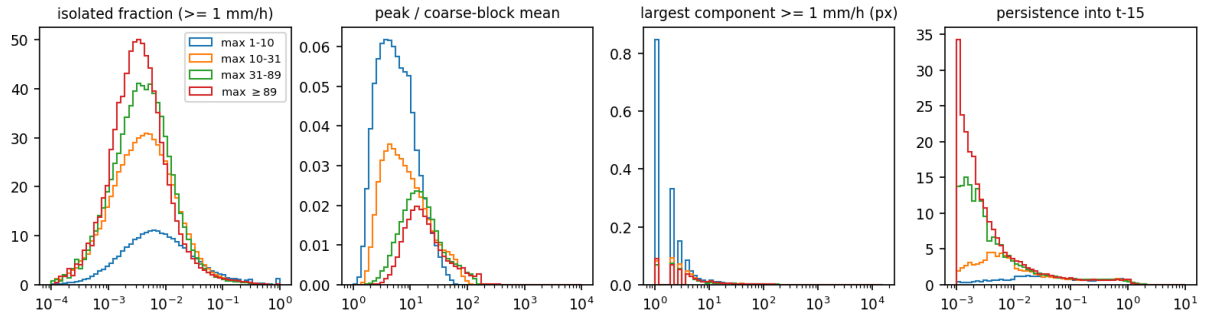


Figure 10: Four features by raw-max stratum (tiles with  $\max \geq 1$  mm/h; log-spaced bins, density per bin). Persistence has a long low tail (the flicker cases,  $\approx 10\%$  of the tail). Log-spaced bins plotted as density visually exaggerate that tail. The median persistence is 0.53–0.72 in every stratum.

## 13 Limitations

- Rule thresholds are uncalibrated. The rates measure signature prevalence, not artefact counts.
- There is no radar-site geometry, so spokes and rings merged with rain are missed (Fig. 5e, Fig. 8, rows 0 and 5). Addressed in Part II: rays through radar sites (§15) and climatological rings (§16).
- The in-cell criterion for  $> 150$  mm/h pixels is fooled by extended artefacts (Fig. 7, bottom).
- Flag rates are correlated across tiles of the same storm, so the standard errors are optimistic.
- QIND exists only in the fetched stores, and even there it is missing on much of the domain.
- Tiles with partial coverage are excluded, following the current patch definition. The domain edge is therefore under-represented.

## Part II

# After the audit: the v2 screen and its validation (28 September – 3 October 2026)

Part I *described* the archive. Part II records what was built on it, and how the result was checked against independent data rather than against labels. Two things changed the approach.

- **No labelled set** (DECISIONS §17). The policy is to reject *clear errors* only and keep imperfections. Rules are therefore restricted to signatures that are non-meteorological by construction, and judged by their rejection rate per intensity bin instead of by precision and recall against labels (§8).
- **Rain gauges as the independent truth.** A rule that removes artefacts should flag pixels that a gauge underneath sees as dry far more often than the pixels it leaves alone, *at the same radar intensity*. §18 measures this for every rule.

The day-to-day record is in §5 of EXPERIMENTS.md, and the plan this follows is §7.4b of RESEARCH\_NOTES.md.

## 14 Corrections to Part I

**The DEM was N-S mirrored (found 28 September).** The DEM GeoTIFF is north-first and the precipitation stores are south-first, and both were sliced with the same offsets. Every DEM-derived feature in Part I (`sea_frac`, `dem_mean`, `argmax_over_sea`, `wet_over_sea_frac`) and the `sea_clutter` rule were computed on the terrain of the mirrored place. The feature files are from 23 September and have not been recomputed, so those columns, and the `sea_clutter` rates in Fig. 9, are wrong. Nothing else in Part I depends on the DEM. Grid orientation and coordinates now come from `src/data/geo.py`.

**The 150 mm/h cap is gone from the archive.** Fig. 2 showed that the original 2023–24 stores never exceed 150 mm/h. All 457 days from 2023-08-01 to 2024-10-30 have since been replaced by archive stores with QIND (checked on disk, 3 October). The archive copies agree with the originals below 149 mm/h and add the values above. The originals of the 101 days after the product switch are kept in `raw/OPERA_orig_postswitch/`, for the paired product comparison below.

**A product break on 2024-07-05.** OPERA switched from ODYSSEY (QIND\_RATE, a composite of the scans in a 15-minute window) to NIMBUS (RATE, the lowest-elevation scan at the nominal time). On days held in both products, NIMBUS has 30–40% less wet area, median intensities 8–20% higher, half the +15 min persistence, and a log-space correlation with ODYSSEY of only 0.65–0.80. v2 therefore trains and tests on ODYSSEY only, and keeps NIMBUS as a separate product-shift split (DECISIONS §18). Every result below is reported per product, because the two differ in exactly the quantities the rules use.

## 15 The v2 cleaning

`src/data/cleaning.py` is used identically by the tile scan, the store build and the gauge pairing, so what is selected, what is stored and what is validated cannot drift apart (tests: `tests/test_cleaning.py`). It has two levels.

**Pixel repairs** (point-like clear errors). Nothing is set to zero to hide a value; a repaired pixel takes the value its neighbourhood supports.

- *Drizzle*:  $R < 0.1$  mm/h  $\rightarrow 0$ , the noise floor, as in v1.
- *Static clutter*: a pixel is *hot* in year  $y$  if  $\hat{p}_{31} > 0.01$  over that year’s valid steps (about  $100\times$  climatology; the climatology of §5, now computed per calendar year over the whole archive). A hot pixel takes the median of the non-hot valid pixels of its  $5\times 5$  neighbourhood, if that is lower.
- *Spikes*: a pixel with  $R \geq 10$  mm/h whose brightest 8-neighbour is below 10% of it has no spatial support at 2 km, and is lowered to that neighbour’s value.

Values between 150 and 500 mm/h are kept as measured: v2 has no declutter zeroing.

**Tile rejections** (extended clear errors).

- *Unphysical*: cleaned maximum  $> 500$  mm/h (about 67 dBZ with Marshall–Palmer). Provisional; the bound is to be set with MeteoSwiss.
- *Ray*: a thin straight component at least 40 px (80 km) long, within  $10^\circ$  of the direction to an OPERA radar within 250 km (RLAN interference). The alignment with a radar is what separates a ray from a squall line, which is also long and thin. This answers the “no radar-site geometry” limitation of Part I for rays.

Rejection is light below the unphysical bound (Table 1). The resulting datasets: 1,977,908 train, 251,667 validation, 282,684 test and 487,743 NIMBUS patches, on splits drawn by whole ISO weeks with at least 2 days between train and validation or test, and 26 catalogued events forced into test.

| raw tile max (mm/h) | tiles      | rejected | unphysical | ray   |
|---------------------|------------|----------|------------|-------|
| [1, 10)             | 13,347,882 | 0.16%    | 0.00%      | 0.16% |
| [10, 31)            | 4,743,452  | 0.44%    | 0.00%      | 0.44% |
| [31, 89)            | 2,129,084  | 0.39%    | 0.00%      | 0.39% |
| [89, 150)           | 450,892    | 0.26%    | 0.00%      | 0.26% |
| [150, 500)          | 316,681    | 0.44%    | 0.00%      | 0.44% |
| [500, $\infty$ )    | 100,987    | 40.45%   | 40.33%     | 0.36% |

Table 1: Tile rejection by raw maximum in the v2 build (OPERA/v2/report.md). Below 500 mm/h only the ray rule rejects. Above it, 60% of tiles survive because the pixel repairs bring their maximum under the bound.

## 16 Audit flags: temporal support and range rings

Two further signatures are computed as *flags* that remove nothing until the gauges have judged them. `scripts/dataset_v2/scan_flags.py` writes them for every covered tile with cleaned maximum  $\geq 1$  mm/h (13.2 h on 6 workers over 5,023 days), together with the position of the tile’s maximum, the nearest active radar and the QIND there.

**Definition 1** (Temporal support). Let  $C$  be an 8-connected cell of the cleaned frame with  $R_t \geq 10$  mm/h, and  $W(C)$  the pixels within 15 px (30 km, a 15-minute move at 33 m/s) of  $C$  in a square window.  $C$  is *unsupported* if

$$\max_{W(C)} R_{t-15} < 1 \quad \text{and} \quad \max_{W(C)} R_{t+15} < 1 \text{ mm/h,}$$

i.e. it has neither a precursor nor a successor. It is *undecidable* if either neighbour frame is missing or not fully covered over  $W(C)$ . Neighbour frames at 00:00 and 23:45 are read from the adjacent day stores, and only if exactly 15 min away.

**Range rings.** Rings are not thin arcs in single frames: a single-frame detector (`cleaning.ring_flag`, a circle about a radar fitting clearly better than a line) found none on 7,005 tiles. They are sparse pixels on a circle that only add up over time, so `ring_climatology.py` detects them in the per-year exceedance climatology. With static-clutter pixels set to NaN, and  $f = \hat{p}_{31}$  averaged per 1-px range bin and  $10^\circ$  sector about each radar site (current and archive entries, ranges 16–150 px), a range  $r$  is a ring in a sector if

$$f(r) > 3 \text{ median}\{f(r') : 3 \leq |r' - r| \leq 6\} \quad \text{and} \quad f(r) > 5 \cdot 10^{-5},$$

and a ring if this holds in at least 18 of the 36 sectors ( $180^\circ$ ). The mask covers the ring range  $\pm 1$  px in its hit sectors. Found: Stevns and Sindal (DK) at about 234 km in every year 2013–2017, and Ikaalinen (178 km) and Vimpeli (49 km) in Finland in 2012; none after the September 2017 compositing change. A tile is `on_ring` when its maximum lies on the mask.

## 17 Consistently bad radars

`radar_quality_v2.py` attributes every pixel to the nearest radar active that year within 250 km, and scores each radar-year on six signals:

- from the climatology: the hot-pixel share, and the exceedance frequencies  $f_{31}$ ,  $f_{150}$ ,  $f_{500}$  over non-hot pixels, each divided by the median of the 5 nearest other radars that year, since rain climates differ (`rel_f*`);
- from the flags: the share of tail tiles ( $\geq 31$  mm/h) whose maximum is unsupported, or on a ring (radar-years with at least 50 tail tiles).

A radar-year is an *outlier* on a signal if it is in the worst 5% of radar-years *and* above zero. A radar is *consistently bad* if it is an outlier on the same signal in at least half of its years, and at least 3.

| radar | site            | country | years | bad on                           |
|-------|-----------------|---------|-------|----------------------------------|
| frbol | Bollène         | FR      | 15    | hot pixels                       |
| seang | Ängelholm       | SE      | 15    | unsupported                      |
| itteo | Teolo           | IT      | 13    | $f_{500}$                        |
| isegs | Fljotsdalsheidi | IS      | 13    | hot pixels, $f_{31}$ , $f_{150}$ |
| –     | Monte Lauro     | IT      | 10    | $f_{31}$ , $f_{500}$             |
| dkste | Stevns          | DK      | 10    | unsupported                      |
| dksin | Sindal          | DK      | 10    | unsupported                      |
| sekk  | Karlskrona      | SE      | 7     | unsupported, ring                |
| sease | Hemse           | SE      | 6     | unsupported                      |
| grand | Andraveda       | GR      | 6     | $f_{31}$ , $f_{150}$             |
| hrdeb | Debeljak        | HR      | 5     | hot pixels                       |
| sehud | Hudiksvall      | SE      | 5     | $f_{31}$                         |
| debln | Berlin          | DE      | 4     | ring                             |
| sevar | Vara            | SE      | 4     | $f_{31}$ , unsupported, ring     |

Table 2: The 14 consistently bad radars of 261 (3,071 radar-years, 2012–2026). “Years” counts the radar-years in the ranking. Nothing is excluded on this list yet.

**Three bugs in the first run.** The first run with flags listed 204 of 246 radars.

- The ring share is zero in 73% of radar-years, so its 95th percentile was 0 and the test  $\geq q_{0.95}$  flagged 75.5% of them; hence the “above zero” condition.
- Nine sites have both a current and an archive entry in the radar database with overlapping years (e.g. Flechtendorf, Essen, Rostock). Both counted as active: they split the site’s pixels, appeared twice per year, and were each other’s nearest neighbour in the relative scores. Radars are now keyed by ODIM code with duplicates merged.

- Sixteen sites have no ODIM code (e.g. Monte Lauro). The ranking dropped them, and the flag scan writes the same empty label for them as for “no radar within 250 km”. They are now keyed by location, and flagged tiles are re-attributed from the position of their maximum.

**Attribution caveat.** The database lists Stevns and Sindal only from 2017, so their 2013–16 rings fall on the nearest radar active then. Ängelholm has a non-zero ring share only in 2013–17 and is probably carrying Stevns’s ring. The ring signal cannot be used per radar until those archive years are added.

## 18 Validation against rain gauges

**Gauges.** 10-minute records from DWD (1,454 stations with data) and MeteoSwiss Swiss-MetNet (281), all open (`scripts/validation/fetch_gauges.py`). They cover Germany and Switzerland only: CombiPrecip for 2012–2026 has been requested from MeteoSwiss, and EU-RADCLIM needs a KNMI key. The gauges are first checked against each other, never against the radar (`gauge_qc.py`). Values above 50 mm per 10 min (including the 99.9 and 149.6 fault codes), runs of at least 6 identical values  $\geq 1$  mm, and bursts  $\geq 10$  mm that have exactly 0 before and after *and* no rain at any gauge within 30 km over  $\pm 30$  min are masked. That is 230 of 84.7M wet values.

**Pairing.** `gauge_vs_radar.py` cleans each frame on a crop around the gauges, with a 40-px margin so the cleaned values equal those of the full frame. For every gauge with a complete record that day, it keeps one row per frame where the raw  $3 \times 3$  radar maximum reaches 5 mm/h or a nearby gauge interval reaches 0.5 mm per 10 min. Each row carries the raw and cleaned values at the gauge pixel, the hot, temporal-support and ring flags there, QIND, and the gauge totals of the 10-minute intervals labelled  $t - 10$ ,  $t$ ,  $t + 10$  and  $t + 20$ . In all, 12,091,733 station-frames from 4,968 days.

**Alignment.** The interval labelling is left open and read from the data. The log-rate correlation peaks at the intervals labelled  $t + 10$  (0.29–0.47) and  $t + 20$  in all four network  $\times$  product cells (Table 3). The matched gauge rate is the larger of those two intervals, in mm/h.

| network | product | $n$       | $t - 10$ | $t$  | $t + 10$    | $t + 20$ |
|---------|---------|-----------|----------|------|-------------|----------|
| DWD     | ODYSSEY | 6,352,421 | −0.04    | 0.16 | <b>0.45</b> | 0.26     |
| DWD     | NIMBUS  | 1,312,918 | −0.01    | 0.17 | <b>0.47</b> | 0.23     |
| SMN     | ODYSSEY | 2,330,589 | −0.04    | 0.14 | <b>0.32</b> | 0.15     |
| SMN     | NIMBUS  | 398,360   | −0.05    | 0.08 | <b>0.29</b> | 0.15     |

Table 3: Correlation of  $\log(1 + R)$  between the radar and the gauge interval with each label.

**Definition 2** (Corroboration). A radar pixel is *corroborated* if the matched gauge rate is  $\geq 1$  mm/h: it rained there. It is *confirmed heavy* if the gauge rate is  $\geq 10$  mm/h. Artefacts (clutter, spikes, rings, interference) leave the gauge dry; a real cell rarely does, despite the point-versus-2km and timing mismatches. A rule is informative if the pixels it flags are corroborated much less often than untouched pixels *in the same raw-intensity bin*.

**Results** (Fig. 11; `validation/corroboration.csv`).

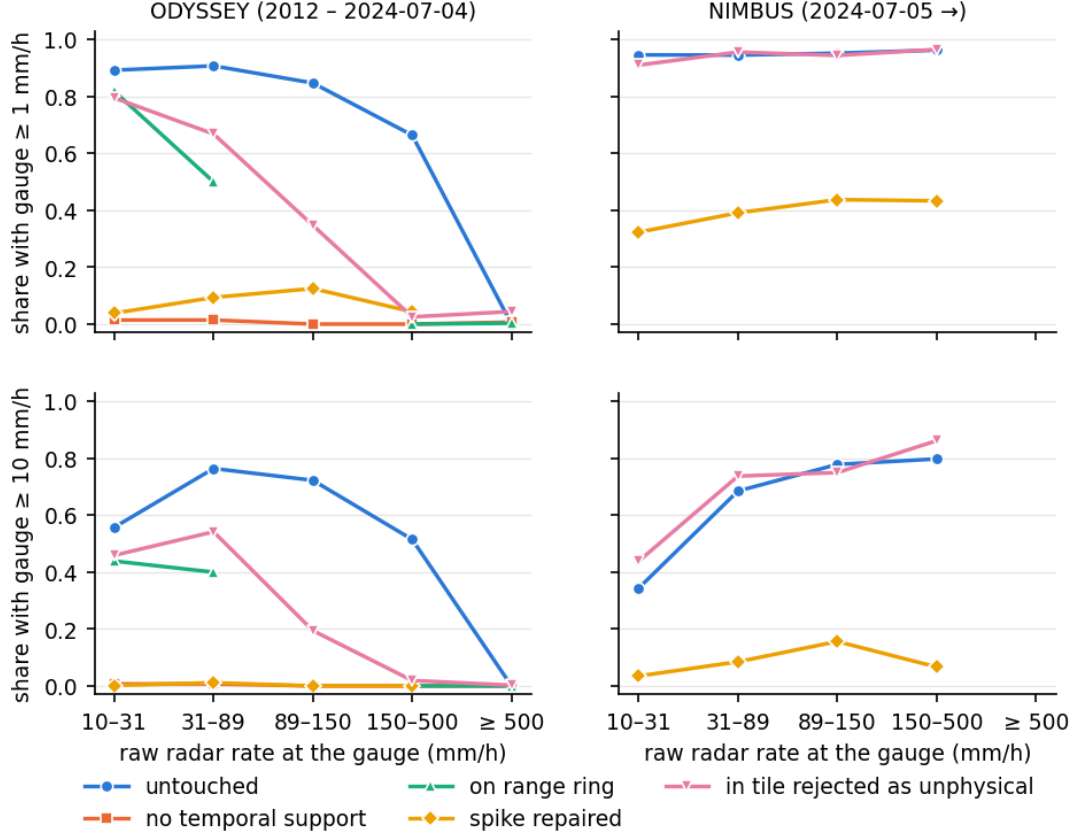


Figure 11: Share of radar pixels corroborated by the gauge underneath (top) and confirmed heavy (bottom), by raw radar rate, per product. “In tile rejected as unphysical” covers every gauge pixel inside a rejected tile, not only its maximum. Bins with fewer than 20 pairs are omitted; under NIMBUS, temporal support and rings almost never fire (2 pixels and 0). [notes/figures/make\\_validation\\_figures.py](#).

1. **Untouched pixels are mostly real.** 0.85–0.91 corroborated under ODYSSEY from 10 to 150 mm/h, and 0.95–0.96 under NIMBUS in every bin up to 500. The exception is the ODYSSEY upper tail: 0.66 corroborated at 150–500 mm/h ( $n = 134$ , median gauge 11 mm/h), against 0.96 under NIMBUS ( $n = 406$ , median 28 mm/h). The 34 untouched pixels above 500 mm/h are all uncorroborated. By year, the untouched tail ( $\geq 31$ ) is 0.77–0.93 corroborated in 2013–19 and 0.92–0.95 from 2020 (2012: 0.36,  $n = 14$ ).
2. **Temporal support is the cleanest signature.** About 1% of unsupported pixels are corroborated, in every bin. It fires almost only under ODYSSEY.
3. **Rings carry real rain at low rates only.** 0.82 corroborated at 10–31 mm/h, 0.50 at 31–89, and 0 at 89 mm/h and above ( $n = 1,782$ ). Real rain falls under the ring; the high values on it are the artefact.
4. **The spike rule depends on the product.** Under ODYSSEY, 4–13% of repaired pixels are corroborated, so it removes artefacts. Under NIMBUS, 32–44%: the instantaneous product resolves small real cores that the rule takes for spikes.
5. **Hot-pixel repair is right.** 19,193 hot pixels above 500 mm/h: 22% are corroborated, but only 0.6% confirmed heavy (median gauge 0.06 mm/h). The clutter coincides with drizzle, not with storms.

6. **The unphysical rule rejects real rain around its artefact.** Gauge pixels inside rejected tiles are 0.67–0.79 corroborated below 89 mm/h under ODYSSEY, and 0.91–0.97 in *every* bin under NIMBUS. Under NIMBUS, a rejected tile is a real storm with one bad pixel.
7. **The cleaning removes little rain.** Of 19,904 pixels lowered by more than 50% from  $\geq 31$  mm/h, the gauge saw  $\geq 10$  mm/h at 155 (0.8%).
8. **The radar underestimates heavy rain.** For gauge rates  $\geq 30$  mm/h (117,411 station-frames), the cleaned  $3 \times 3$  radar maximum is  $\geq 10$  mm/h in 71% of cases, with a median radar/gauge ratio of 0.45.
9. **QIND does not discriminate.** Among raw  $\geq 31$  mm/h, the AUC of QIND for corroborated against not is 0.53 under NIMBUS (both medians 1.00). Under ODYSSEY it is 0.26: *inverted*, with median QIND 0.20 for corroborated pixels and 0.90 for uncorroborated ones. The inversion is unexplained; clutter close to a radar may get a high quality score. Check it before using QIND even as a covariate.

**What the numbers suggest** (proposals; the decisions are pending, EXPERIMENTS §5).

- Apply temporal support: reject or mask unsupported cells.
- Rings: repair the high values on the ring instead of rejecting the tile.
- Spikes: keep the rule for ODYSSEY; revisit it, or its thresholds, for NIMBUS.
- Unphysical: mask the offending pixels instead of rejecting the tile, at least under NIMBUS.
- ODYSSEY 150–500 mm/h: a third of it is uncorroborated even after cleaning. The POT threshold and any tail claim should be checked with and without that range.

#### Limits of this validation.

- Germany and Switzerland only. Of the radars in Table 2, only Berlin is inside that area.
- Corroboration at 1 mm/h shows that it rained, not that a 300 mm/h value is right; the  $\geq 10$  mm/h row is the stronger test.
- A gauge samples a point, the radar a 2 km pixel, and ODYSSEY integrates over a 15-minute window, so even real cells are not always corroborated (the ceiling is about 0.9).
- Pairs from the same storm are correlated, so the effective sample sizes are smaller than the  $n$  shown. Most flagged classes above 89 mm/h hold only tens to hundreds of pairs.

## Reproducing

```
# whole audit, CPU only, cores 4-11 (see EXPERIMENTS.md §0)
setuid bash scripts/hpc/launch_data_quality.sh \
    /home/fquareng/work/data/extremes/OPERA/quality 6 &
# after labelling:
python scripts/data_quality/summarize_features.py config.yaml \
    --features_dir ../quality/features --out_dir ../quality/summary \
    --labels ../quality/gallery/random_max31-inf/labels.csv
# figures in these notes:
python notes/figures/make_data_quality_figures.py
python notes/figures/make_validation_figures.py

# Part II (see each script's docstring for its arguments)
bash scripts/dataset_v2/run_scan.sh          # per-year climatology + tile scan
bash scripts/dataset_v2/run_build.sh         # splits, store, gamma, checks
python scripts/data_quality/ring_climatology.py
```

```
bash scripts/dataset_v2/run_flags.sh      # temporal support, rings
# gauges, pairs, tail validation, radar ranking:
setsid nohup bash scripts/validation/run_validation.sh \
    > logs/validation.log 2>&1 &
```